LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

MODUL 4&5

Core Of OOP & Encapsulation

Disusun Oleh:

Bagas Efriliyanto Setiawan (105224049)

PROGRAM STUDI ILMU KOMPUTER

FAKULTAS SAINS DAN KOMPUTER

UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Pengembangan arsitektur sistem reservasi pada jaringan manajemen perhotelan saat ini menjadi prioritas utama guna mengatasi berbagai kendala operasional yang sering terjadi, seperti ketidakteraturan kuota tamu di dalam kamar hingga potensi penyalahgunaan kode voucher oleh pelanggan. Dalam proyek perancangan ini, fokus utama terletak pada pembuatan blueprint kelas KamarHotel yang mengintegrasikan prinsip Encapsulation secara ketat untuk memastikan bahwa seluruh data krusial seperti nomor kamar, tipe, kapasitas, hingga harga tidak dapat dimodifikasi secara ilegal dari luar sistem. Selain pengamanan data, sistem ini juga dirancang untuk memiliki kecerdasan dalam memvalidasi input secara otomatis melalui penggunaan Constructor ganda dan metode Setter yang dilengkapi logika bisnis. Dengan menerapkan konsep Method Overloading, sistem mampu memberikan respons yang adaptif terhadap berbagai skenario pemesanan dan penagihan, sehingga menciptakan ekosistem manajemen hotel yang lebih tangguh, minim bug, dan transparan bagi divisi keuangan maupun operasional.

II. Variabel

No	Nama Variabel	Tipe Data	Fungsi
1	nomorKamar	String	Menyimpan identitas unik berupa nomor kamar hotel.
2	tipeKamar	String	Menyimpan kategori kamar (Reguler, Premium, atau Suite).
3	kapasitasMaksimal	int	Menentukan batas jumlah tamu maksimal yang diizinkan dalam satu kamar.
4	hargaPerMalam	double	Menyimpan tarif sewa kamar per malam dengan batas minimal 50.000.
5	isTersedia	boolean	Indikator status kamar, bernilai true jika kosong dan false jika sudah dipesan.

III. Dokumentasi dan Pembahasan Code

```java
public class KamarHotel {
    // Atribut dibuat private biar gak bisa diakses sembarangan dari luar (Encapsulation)
    private String nomorKamar;
    private String tipeKamar;
    private int kapasitasMaksimal;
    private double hargaPerMalam;
    private boolean isTersedia;

    // Constructor 1: Buat yang mau daftar cepet (Pendaftaran Kilat)
    public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal) {
        this.nomorKamar = nomorKamar;
        setTipeKamar(tipeKamar); // Panggil setter biar validasi tipe tetep jalan
        this.kapasitasMaksimal = kapasitasMaksimal;
        this.hargaPerMalam = 0;
        this.isTersedia = true;
    }

    // Constructor 2: Pendaftaran yang datanya lengkap
    public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal, double hargaPerMalam) {
        this.nomorKamar = nomorKamar;
        setTipeKamar(tipeKamar);
        this.kapasitasMaksimal = kapasitasMaksimal;
        setHargaPerMalam(hargaPerMalam); // Panggil setter biar validasi harga tetep jalan
        this.isTersedia = true;
    }

    // Getter untuk semua atribut
    public String getNomorKamar() { return nomorKamar; }
    public String getTipeKamar() { return tipeKamar; }
    public int getKapasitasMaksimal() { return kapasitasMaksimal; }
    public double getHargaPerMalam() { return hargaPerMalam; }
    public boolean isTersedia() { return isTersedia; }

    // Setter untuk Tipe Kamar dengan validasi "Reguler", "Premium", atau "Suite"
    public void setTipeKamar(String tipeKamar) {
        if (tipeKamar.equalsIgnoreCase("Reguler") ||
            tipeKamar.equalsIgnoreCase("Premium") ||
            tipeKamar.equalsIgnoreCase("Suite")) {
            this.tipeKamar = tipeKamar;
        } else {
            System.out.println("Peringatan: Tipe tidak dikenal, otomatis diatur ke Reguler.");
            this.tipeKamar = "Reguler";
        }
    }

    // Setter untuk Harga dengan batas minimal 50.000
    public void setHargaPerMalam(double hargaPerMalam) {
        if (hargaPerMalam < 50000) {
            System.out.println("Peringatan: Harga terlalu rendah, otomatis diatur ke 50000.");
            this.hargaPerMalam = 50000;
        } else {
            this.hargaPerMalam = hargaPerMalam;
        }
    }

    // Method Overloading 1: Pesan kamar tanpa parameter
    public void pesanKamar() {
        if (isTersedia) {
            isTersedia = false;
            System.out.println("Kamar " + nomorKamar + " berhasil dipesan.");
        } else {
            System.out.println("Peringatan: Kamar sudah terisi!");
        }
    }

    // Method Overloading 2: Pesan kamar dengan jumlah tamu
    public void pesanKamar(int jumlahTamu) {
        if (!isTersedia) {
            System.out.println("Peringatan: Kamar sudah terisi!");
        } else if (jumlahTamu > kapasitasMaksimal) {
            System.out.println("Gagal: Kapasitas tidak cukup untuk " + jumlahTamu + " orang!");
        } else {
            isTersedia = false;
            System.out.println("Kamar " + nomorKamar + " berhasil dipesan untuk " + jumlahTamu + " orang.");
        }
    }

    public void batalPesan() {
        isTersedia = true;
        System.out.println("Pesanan kamar " + nomorKamar + " telah dibatalkan.");
    }

    // Method Overloading 3: Hitung bayar standar
    public double hitungTotalBayar(int jumlahMalam) {
        return hargaPerMalam * jumlahMalam;
    }

    // Method Overloading 4: Hitung bayar pakai voucher "PROMO"
    public double hitungTotalBayar(int jumlahMalam, String kodeVoucher) {
        double total = hitungTotalBayar(jumlahMalam);
        if (kodeVoucher.equalsIgnoreCase("PROMO") && jumlahMalam >= 3) {
            return total * 0.8; // Diskon 20%
        } else {
            System.out.println("Voucher ditolak: Kode salah atau durasi inap kurang dari 3 malam.");
            return total;
        }
    }

    // Cetak profil kamar biar rapi
    public void tampilkanProfil() {
        System.out.println("--- INFORMASI KAMAR ---");
        System.out.println("Nomor     : " + nomorKamar);
        System.out.println("Tipe      : " + tipeKamar);
        System.out.println("Kapasitas : " + kapasitasMaksimal + " orang");
        System.out.println("Harga     : Rp" + hargaPerMalam);
        System.out.println("Tersedia  : " + (isTersedia ? "Ya" : "Tidak"));
        System.out.println("-----------------------");
    }
}
```

Gambar 1. Keseluruhan Kode KamarHotel.

1. Bedah Kelas KamarHotel.java (Logika Bisnis)

Di sini kita nerapin Encapsulation dan Overloading sesuai permintaan soal:

- Atribut Private (Baris 3-7): Semua variabel kayak nomorKamar sampai isTersedia kita kasih akses private. Tujuannya biar nggak bisa diubah sembarangan dari luar kelas tanpa lewat "pintu" yang resmi.

- Dual Constructor (Baris 10-25): * Pertama: Buat daftar kilat, cuma butuh nomor, tipe, sama kapasitas. Harganya otomatis jadi 0.

    - Kedua: Buat daftar lengkap yang nerima input harga juga. Keduanya otomatis set status kamar jadi true (tersedia).

- Validasi Setter (Baris 35-55): Ini bagian yang paling krusial.

    - Di setTipeKamar(), kalau inputnya bukan Reguler, Premium, atau Suite, sistem bakal nolak dan maksa jadi "Reguler".

    - Di setHargaPerMalam(), kalau harganya di bawah 50.000, langsung dipaksa jadi 50.000.

- Method Overloading Pesan Kamar (Baris 58-75): * pesanKamar() tanpa parameter langsung ubah status jadi false kalau tersedia.

    - pesanKamar(int jumlahTamu) punya logika tambahan: kalau tamu lebih banyak dari kapasitas, pesanan ditolak dan kamar tetep true.

- Logika Voucher (Baris 83-93): Kita pakai overloading lagi buat hitung bayar. Kalau kodenya "PROMO" dan nginep minimal 3 malam, baru dapet diskon 20%. Kalau nggak memenuhi syarat, diskon hangus.

```java
public class Main {
    public static void main(String[] args) {
        // Test Kasus 1: Kamar dengan input asal-asalan (Presidential & harga minus)
        KamarHotel kamar1 = new KamarHotel("101", "Presidential", 2, -10000);

        // Test Kasus 2: Kamar kedua dengan pendaftaran kilat
        KamarHotel kamar2 = new KamarHotel("202", "Suite", 2);
        kamar2.setHargaPerMalam(1000000);

        // Uji coba pesan kamar kedua dengan tamu berlebih (Overcapacity)
        System.out.println("\n--- Simulasi Pemesanan Kamar 2 ---");
        kamar2.pesanKamar(4); // Harusnya gagal
        kamar2.pesanKamar(2); // Harusnya sukses
        kamar2.pesanKamar();  // Harusnya dibilang sudah terisi

        // Uji coba hitung tagihan
        System.out.println("\n--- Simulasi Pembayaran ---");

        // Kamar 1: Coba voucher PROMO tapi cuma 2 malam (Harus ditolak diskonnya)
        System.out.println("Kamar 1 (2 malam, kode PROMO):");
        double tagihan1 = kamar1.hitungTotalBayar(2, "PROMO");
        System.out.println("Total Bayar: Rp" + tagihan1);

        // Kamar 2: Coba voucher PROMO selama 4 malam (Harus dapet diskon 20%)
        System.out.println("\nKamar 2 (4 malam, kode PROMO):");
        double tagihan2 = kamar2.hitungTotalBayar(4, "PROMO");
        System.out.println("Total Bayar: Rp" + tagihan2);

        // Cetak Struk Akhir
        System.out.println("\n=== STRUK INFORMASI AKHIR ===");
        kamar1.tampilkanProfil();
        kamar2.tampilkanProfil();
    }
}
```

Gambar 2. Keseluruhan Kode Main

2. Bedah Kelas Main.java (Simulasi Uji Coba)

Ini bagian buat ngebuktiin kalau kode kita "kebal" sama kesalahan operasional:

- Uji Ketahanan Setter (Kamar 1): Kita sengaja masukin tipe "Presidential" dan harga "-10000". Pas di-run, sistem otomatis benerin jadi "Reguler" dan "50000".
- Uji Overcapacity (Kamar 2): Kamar ini kapasitasnya cuma 2 orang, tapi kita coba pesan buat 4 orang pakai pesanKamar(4). Sistem terbukti ngeblokir pesanan itu.
- Uji Double Booking: Setelah berhasil pesan dengan jumlah tamu yang bener, kita coba pesan lagi. Sistem bakal kasih peringatan kalau kamar sudah terisi.
- Uji Voucher: * Kamar 1 gagal dapet diskon karena cuma 2 malam (kurang dari syarat minimal).
    - Kamar 2 sukses dapet potongan 20% karena nginep 4 malam pakai kode "PROMO".
- Struk Akhir: Terakhir kita panggil metode buat nyetak informasi lengkap biar rapi sesuai permintaan divisi keuangan.

```
Peringatan: Tipe tidak dikenal, otomatis diatur ke Reguler.
Peringatan: Harga terlalu rendah, otomatis diatur ke 50000.

--- Simulasi Pemesanan Kamar 2 ---
Gagal: Kapasitas tidak cukup untuk 4 orang!
Kamar 202 berhasil dipesan untuk 2 orang.
Peringatan: Kamar sudah terisi!

--- Simulasi Pembayaran ---
Kamar 1 (2 malam, kode PROMO):
Voucher ditolak: Kode salah atau durasi inap kurang dari 3 malam.
Total Bayar: Rp100000.0

Kamar 2 (4 malam, kode PROMO):
Total Bayar: Rp3200000.0

=== STRUK INFORMASI AKHIR ===
--- INFORMASI KAMAR ---
Nomor     : 101
Tipe      : Reguler
Kapasitas : 2 orang
Harga     : Rp50000.0
Tersedia  : Ya
-----------------------
--- INFORMASI KAMAR ---
Nomor     : 202
Tipe      : Suite
Kapasitas : 2 orang
Harga     : Rp1000000.0
Tersedia  : Tidak
-----------------------
PS C:\Users\lenovo\Documents\BAGAS\PBO\Modul 5\THT Modul 4&5\src>
```

1. Keamanan Pintu Masuk (Setter Validation)

Di bagian awal, sistem langsung mendeteksi input yang "asal-asalan" pada objek Kamar 1 (nomor 101).

- Karena kita mencoba memasukkan tipe kamar di luar pilihan, muncul pesan: "Peringatan: Tipe tidak dikenal, otomatis diatur ke Reguler".
- Begitu juga dengan harga yang kita set di bawah batas minimum, sistem langsung bereaksi dengan: "Peringatan: Harga terlalu rendah, otomatis diatur ke 50000".

2. Simulasi Operasional (Overloading Pesan Kamar) Pada blok simulasi Kamar 2 (nomor 202), kita menguji validasi kapasitas.

- Gagal Overcapacity: Saat dipesan untuk 4 orang (padahal kapasitas cuma 2), sistem menolak dengan pesan "Gagal: Kapasitas tidak cukup untuk 4 orang!".
- Berhasil: Begitu jumlah tamunya sesuai (2 orang), sistem mencetak "Kamar 202 berhasil dipesan".
- Double Booking: Saat coba dipesan lagi, sistem konsisten memberikan peringatan "Kamar sudah terisi!".

3. Simulasi Keuangan (Overloading Hitung Bayar & Voucher) Di sini kita menguji logika pemberian diskon yang sangat spesifik.

- Voucher Ditolak: Pada Kamar 1, meskipun menggunakan kode "PROMO", diskon tidak diberikan karena hanya menginap 2 malam (syarat minimal 3 malam). Total bayarnya tetap harga normal (Rp100.000).
- Voucher Berhasil: Pada Kamar 2, karena menginap 4 malam dengan kode "PROMO", diskon 20% otomatis memotong tagihan menjadi Rp3.200.000 dari harga normal Rp4.000.000.

4. Struk Informasi Akhir Sebagai penutup, sistem menampilkan profil lengkap kedua kamar dengan rapi.

- Kita bisa lihat di situ status Tersedia: Ya untuk Kamar 1 karena memang belum dipesan.
- Sedangkan Kamar 2 berstatus Tersedia: Tidak karena proses pemesanan sebelumnya sudah berhasil dikunci oleh sistem.

IV. Kesimpulan

Secara keseluruhan, perancangan sistem reservasi ini berhasil membuktikan bahwa integrasi antara keamanan data dan fleksibilitas metode adalah kunci dalam membangun aplikasi yang reliabel. Melalui penerapan Encapsulation, sistem terbukti mampu menolak input yang tidak sesuai standar, seperti tipe kamar di luar kategori resmi atau harga yang berada di bawah batas minimum operasional, dan secara otomatis melakukan koreksi nilai demi menjaga konsistensi database. Penggunaan Method Overloading juga memberikan keunggulan pada aspek fungsionalitas, di mana sistem dapat memvalidasi kapasitas kamar terhadap jumlah tamu secara real-time serta menghitung tagihan promo dengan akurasi tinggi berdasarkan durasi menginap pelanggan. Dengan keberhasilan simulasi pada berbagai skenario kesalahan operasional, sistem ini siap digunakan untuk meminimalisir risiko kerugian perusahaan dan meningkatkan efisiensi layanan pada divisi pemesanan serta keuangan.

V. Daftar Pustaka

Oracle. (n.d.). *Scanner (Java Platform SE 8)*. Oracle Documentation.

Oracle. (n.d.). *Assignment, Arithmetic, and Unary Operators*. The Java™ Tutorials.

Schildt, H. (2018). Java: The Complete Reference. McGraw-Hill Education.